



National University of Computer and Emerging Sciences

Laboratory Manuals

for

Computer Networks - Lab

(CL-3001)

Department of Computer Science

FAST-NU, Lahore, Pakistan

Instructor:

Muhammad Nouman Hanif

Lab Manual 07

Objective:

To understand and implement a multithreaded server capable of handling multiple clients concurrently using TCP sockets.

Introduction to Multithreading in Servers

In our previous labs, our servers were **iterative**. An iterative server can only handle one client at a time. While it is engaged with one client, all other clients that try to connect must wait in a queue. This is inefficient for any application that needs to serve multiple users simultaneously, like a chat room or an online game.

The solution is to build a **concurrent server**. A concurrent server can manage multiple client connections at the same time. The most common way to achieve this is with **multithreading**.

What is a Thread?

A thread is the smallest unit of execution within a process. A single process can have multiple threads, all running in parallel and sharing the same memory space. This makes them "lightweight" and ideal for tasks that can run independently, such as handling a client connection.

The Multithreaded Server Model

The model is straightforward:

1. The server's **main thread** starts, creates a listening socket, binds it to an address, and enters an infinite loop.
2. Inside the loop, the main thread calls `accept()`, which blocks until a new client connects.
3. When a client connects, `accept()` returns a new socket object specifically for communicating with that client.
4. Instead of handling the client itself, the main thread spawns a **new worker thread**. It passes the new client socket to this worker thread.
5. The main thread immediately loops back to call `accept()` again, ready to welcome the next client without delay.
6. The worker thread, running in parallel, handles all communication (sending and receiving data) with its assigned client. Once the client disconnects, the worker thread's job is done, and it terminates.

This architecture allows the server to be highly responsive, as the main thread's only job is to accept new connections.

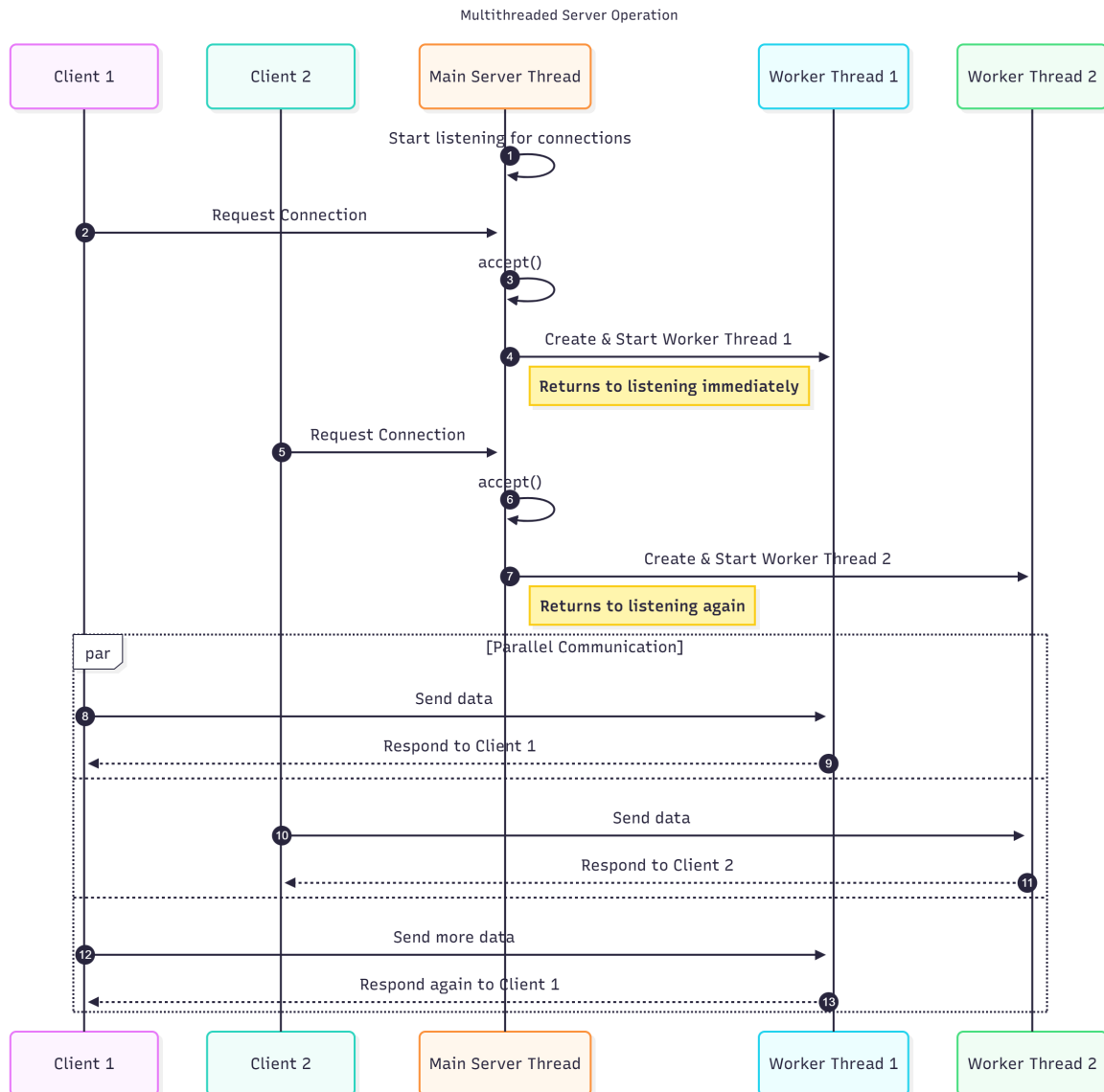


Figure 1: A multithreaded server spawning a new thread for each client.

Python Threading and Server Examples

Python's built-in `threading` module makes it easy to create and manage threads.

1. Simple Thread Creation

This example shows the basic creation and execution of two threads. The `join()` method makes the main program wait until the thread has finished its execution.

```

1 import threading
2
3 # Global thread ID counter and a lock for thread-safe updates
4 tid = 1
  
```

```
5 tid_lock = threading.Lock()
6
7 def print_thread_id():
8     global tid
9     with tid_lock:
10         current_id = tid
11         tid += 1
12     print(f"This is Thread {current_id}")
13
14 def main():
15     # Create two threads
16     thread1 = threading.Thread(target=print_thread_id)
17     thread2 = threading.Thread(target=print_thread_id)
18
19     # Start the threads
20     thread1.start()
21     thread2.start()
22
23     # Wait for both threads to complete
24     thread1.join()
25     thread2.join()
26
27 if __name__ == "__main__":
28     main()
```

Listing 1: Simple_Thread_Creation.py

2. Passing Arguments to a Thread

You can pass arguments to your thread's target function using the `args` parameter in the `Thread` constructor. The arguments must be supplied as a tuple.

```
1 import threading
2
3 def print_thread_id(message):
4     print(message)
5
6 def main():
7     message1 = "Thread 1"
8     message2 = "Thread 2"
9
10    # Create threads and pass arguments
11    thread1 = threading.Thread(target=print_thread_id, args=(message1,))
12    thread2 = threading.Thread(target=print_thread_id, args=(message2,))
13
14    # Start the threads
15    thread1.start()
16    thread2.start()
17
18    # Wait for both threads to finish
19    thread1.join()
20    thread2.join()
21
22 if __name__ == "__main__":
23     main()
```

Listing 2: Thread_Passing_Parameters.py

3. Multithreaded Group Chat Server

This example demonstrates a more practical application of multithreading. The server can handle multiple clients connecting to a chat room. When one client sends a message, the server broadcasts it to all other connected clients. This requires managing a shared list of clients, which must be protected by a `threading.Lock` to ensure thread safety.

Chat Server (chat_server_threaded.py)

```
1 import socket
2 import threading
3
4 clients = []
5 clients_lock = threading.Lock()
6
7 def broadcast(message, sender_conn):
8     """Sends a message to all clients except the sender."""
9     with clients_lock:
10         for client_conn in clients:
11             if client_conn != sender_conn:
12                 try:
13                     client_conn.sendall(message)
14                 except socket.error:
15                     clients.remove(client_conn)
16
17 def handle_client(conn, addr):
18     """Handles a single client, receiving and broadcasting messages."""
19     print(f"[NEW CONNECTION] {addr} connected.")
20     with clients_lock:
21         clients.append(conn)
22
23     try:
24         while True:
25             message = conn.recv(1024)
26             if not message:
27                 break
28             broadcast_message = f"[{addr[0]}:{addr[1]}] {message.decode()}".encode()
29             broadcast(broadcast_message, conn)
30     finally:
31         with clients_lock:
32             clients.remove(conn)
33         print(f"[DISCONNECTED] {addr} disconnected.")
34         conn.close()
35
36 def main():
37     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
38     server.bind(('127.0.0.1', 2001))
39     server.listen()
40     print("[*] Chat server listening on 127.0.0.1:2001")
41     while True:
42         conn, addr = server.accept()
43         thread = threading.Thread(target=handle_client, args=(conn, addr))
44         thread.start()
45
46 if __name__ == "__main__":
47     main()
```

Chat Client (chat_client_threaded.py)

For the chat to be functional, the client also benefits from threading: one thread to listen for incoming messages and the main thread to handle user input for sending messages.

```
1 import socket
2 import threading
3
4 def receive_messages(client_socket):
5     """Continuously receives messages from the server."""
6     while True:
7         try:
8             message = client_socket.recv(1024).decode()
9             if not message:
10                 print("\nDisconnected from server.")
11                 break
12             print(message)
13         except:
14             print("\nConnection to server was lost.")
15             break
16
17 def main():
18     client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
19     client.connect(('127.0.0.1', 2001))
20     print("Connected to chat server! Type and press Enter to send.")
21
22     receive_thread = threading.Thread(target=receive_messages, args=(client,))
23     receive_thread.daemon = True
24     receive_thread.start()
25
26     while True:
27         try:
28             message = input()
29             client.sendall(message.encode())
30         except (KeyboardInterrupt, EOFError):
31             break
32
33     client.close()
34
35 if __name__ == "__main__":
36     main()
```

Lab Exercises

Task 1: Remote Server Status Checker

Create a client-server application where clients can query the server for specific status information. This task demonstrates handling different types of client requests and managing a simple piece of shared state (the client count).

Server Requirements

- The server must be multithreaded.
- It must maintain a shared, thread-safe counter for the number of currently active clients.
- The client handler thread should:
 1. Increment the client counter upon connection and decrement it upon disconnection.
 2. Wait to receive a command string from the client.
 3. Based on the command, send back the appropriate information:
 - If the command is "TIME", respond with the server's current date and time.
 - If the command is "IP", respond with the client's own IP address and port.
 - If the command is "CLIENTS", respond with the total number of currently connected clients.
 4. If the command is not recognized, send back an error message.
 5. The connection should be closed after serving one request.

Client Requirements

- The client should connect to the server.
- It should prompt the user to enter a command (TIME, IP, or CLIENTS).
- It sends the command to the server, receives the response, and prints it to the console before terminating.

Task 2: Online Voting System

You are to implement a secure online vote casting system using a multithreaded TCP server. The server will authenticate voters and record their votes, ensuring that each registered person can vote only once.

Required Files

Before starting, create two text files in the same directory as your server script:

1. **Voters_List.txt:** Contains registered voters. Each line must be in the format **Name/CNIC**.

Muhammad Usama/34569-1245743-3
Haris Masood/36782-2771596-3
Nauman Moazzam Hayat/34095-1889427-1
... (and so on)

2. **Candidates_List.txt:** Contains the candidates and their symbols. Each line should have the candidate's name, a tab character, and then their assigned symbol.

Imran Khan Bat
Nawaz Sharif Tiger
Bilawal Zardari Arrow
... (and so on)

Server Requirements

- The server must be multithreaded to handle simultaneous voters.
- On startup, the server should read and load the contents of **Voters_List.txt** and **Candidates_List.txt** into memory.
- **Thread Safety:** The server must manage shared data. Use a **threading.Lock** to protect:
 - A set or list containing the CNICs of clients who have already voted.
 - A dictionary that stores the vote count for each candidate symbol.
- The handler thread for each voter (client) must perform the following sequence:
 1. Request and receive the client's Name and CNIC number.
 2. **Authentication:** Verify if the CNIC exists in the loaded voters list.
 3. **Vote Check:** Check if the CNIC is already in the shared list of those who have voted. If it is, send an error message and close the connection.
 4. If authenticated and eligible, add the CNIC to the "already voted" list.
 5. Send the list of candidates to the client.
 6. Receive the client's vote (which will be a poll symbol).
 7. Atomically update the vote count for the chosen symbol.
 8. Send a "Vote cast successfully!" confirmation message to the client and close the connection.

Client Requirements

- Connect to the voting server.
- When prompted, send its Name and CNIC to the server.
- If authentication is successful, it will receive the list of candidates. Display this list to the user.
- If authentication fails (or the user has already voted), it will receive an error message. Display it and terminate.
- Prompt the user to enter the poll symbol of their chosen candidate and send it to the server.
- Wait for a final confirmation message from the server, display it, and then terminate.